

FlowForge Guardian

Step-by-step build workflow, coding plan, and working mechanism

Purpose: FlowForge Guardian turns a plain-language business instruction into a testable AI workflow. It runs the workflow in a safe sandbox, checks policy risks, produces a go/no-go report, and asks for human approval before consequential actions.

1. Product at a glance

Stage	What happens
1. Describe	User writes the routine in ordinary language.
2. Compile	LLM converts intent into validated workflow JSON.
3. Visualize	UI displays trigger, checks, actions, and approval gates.
4. Generate tests	System creates realistic edge cases and synthetic records.
5. Simulate	Workflow runs in dry-run mode with no real side effects.
6. Verify	Policy engine checks consent, duplicates, limits, and permissions.
7. Decide	Report gives pass rate, risk, confidence, and fixes.
8. Approve	Human approves, edits, rejects, or postpones activation.
9. Execute	Approved workflow runs against real or connected tools.
10. Audit	Every decision and action is recorded for review.

Core mechanism

```
Natural-language instruction
↓
LLM workflow compiler → JSON schema validator
↓
Workflow graph + policy requirements
↓
Synthetic edge-case generator
↓
Dry-run executor / sandbox
↓
Policy engine + risk scorer + evidence collector
↓
Safety report → human approval → real execution
↓
Audit log + feedback
```

2. Recommended one-week MVP

Day	Deliverable
Day 1	Set up repository, database, app shell, sample lead data, and product specification.
Day 2	Build compiler: natural language to strict JSON with validation and a safe fallback demo workflow.
Day 3	Build workflow graph UI, synthetic test-case generator, and policy rules.
Day 4	Build dry-run simulation engine, event trace, pass/block/fail states, and risk score.
Day 5	Build safety report, approval queue, mock execution, and audit log.
Day 6	Polish UI, animate simulation particles and metrics, test failure states, deploy.

Day 7	Record demo, finalize README, architecture diagram, screenshots, and Devpost submission.
-------	--

3. System architecture

Layer	Technology	Responsibility
Frontend	Next.js + TypeScript + Tailwind + Framer Motion	Builder, graph, simulation lab, safety report, approvals
API	FastAPI or Next.js API routes	Compile, simulate, report, approve, execute
Compiler	LLM + structured output + Zod/Pydantic	Converts intent into workflow JSON
Policy engine	Deterministic Python/TypeScript rules	Blocks or flags unsafe actions
Simulation	Seeded test runner	Runs synthetic records in dry-run mode
Storage	SQLite/Supabase	Workflows, tests, runs, approvals, audit events
Adapters	Mock email/CRM first; real APIs later	Prevents integration delays during MVP

4. Repository structure

```

flowforge-guardian/
├── app/ # Next.js pages and UI
│   ├── page.tsx # Builder
│   ├── simulate/page.tsx # Simulation lab
│   ├── report/page.tsx # Safety report
│   └── runs/page.tsx # Runs and approvals
├── components/
│   ├── WorkflowGraph.tsx
│   ├── SimulationCanvas.tsx
│   ├── RiskMeter.tsx
│   ├── ApprovalCard.tsx
│   └── AuditTimeline.tsx
├── lib/
│   ├── schema.ts
│   ├── compiler.ts
│   ├── policies.ts
│   ├── simulator.ts
│   ├── scorer.ts
│   └── mock-adapters.ts
├── data/sample-leads.json
├── tests/
├── README.md
└── .env.example

```

5. Workflow data model

Use a strict schema so the model cannot directly execute arbitrary actions. The compiler may propose actions, but the validator and policy engine decide what is allowed.

```

type Workflow = {
  id: string; name: string; description: string;
  trigger: { type: "new_lead" | "new_order" | "manual" };
  steps: Step[]; policies: Policy[];
};

type Step =
  | { id: string; type: "validate_email" }
  | { id: string; type: "check_consent" }
  | { id: string; type: "draft_email"; template: string }
  | { id: string; type: "send_email"; requiresApproval: boolean }
  | { id: string; type: "create_task"; dueInDays: number };

```

6. Coding sequence

- A. Create the app shell:** Create Next.js app, add Tailwind and Framer Motion, create navigation, and add mock pages. Do not start with integrations.
- B. Define the schema:** Create workflow, step, policy, test-case, run, approval, and audit-event types. Reject unknown action types.
- C. Compile intent:** Send the instruction to the LLM with a strict JSON-only prompt. Parse and validate it; if validation fails, show a repair message or use a safe demo workflow.
- D. Render graph:** Map each step to a visual node. Draw connectors from ordered steps. Add states: idle, running, passed, blocked, review.
- E. Generate tests:** Create valid lead, missing email, no consent, duplicate, invalid email, excessive discount, and VIP records.
- F. Simulate:** Run every test through every step. Produce an event trace. Never call real integrations during simulation.
- G. Score:** Calculate pass rate, blocked count, failed count, policy violations, risk tier, and confidence.
- H. Approval:** Display exact intended action and data. Approve, reject, or edit. Only approval can move a high-risk action to execution.
- I. Execute mock action:** Write simulated email/task result and audit event. Add real APIs only after the mock path is stable.
- J. Test and deploy:** Run unit tests, manually test states, deploy frontend, and record the demo.

7. Example working mechanism

User instruction: "When a new lead arrives, check consent, draft a welcome email, ask me before sending it, and create a follow-up task in three days."

```
{
  "trigger": {"type": "new_lead"},
  "steps": [
    {"type": "validate_email"},
    {"type": "check_consent"},
    {"type": "draft_email", "template": "welcome"},
    {"type": "send_email", "requiresApproval": true},
    {"type": "create_task", "dueInDays": 3}
  ]
}
```

Step	Event	System behavior
1	New lead enters sandbox	The simulator loads a synthetic lead record.
2	Validate email	Invalid or missing email is blocked; no later email action runs.
3	Check consent	Marketing email without consent is blocked or sent to review.
4	Draft email	A preview is generated, but nothing is sent.
5	Approval gate	User sees recipient, subject, body, and reason for approval.
6	Execute	After approval, mock email and task actions are recorded.
7	Audit	The complete trace is stored and shown in the timeline.

8. Policy engine and scoring

```
function checkPolicies(input, action, context) {
  const violations = [];
  if (action.type === "send_email" && !input.consent)
    violations.push({code: "NO_CONSENT", severity: "high"});
  if (context.sentWithinHours < 24)
    violations.push({code: "DUPLICATE_RISK", severity: "medium"});
  if (input.discount > context.maxDiscount)
    violations.push({code: "DISCOUNT_LIMIT", severity: "high"});
}
```

```
return violations;
}
```

Suggested score:

```
pass_rate = passed_tests / total_tests
risk = high_violations * 3 + medium_violations
confidence = min(100, 60 + coverage_points - unresolved_risks * 10)
activation_allowed = (high_violations == 0)
```

Keep this score clearly labeled as a prototype heuristic. Do not present it as a certified safety or compliance score.

9. Animation plan

Moment	Motion
Workflow generation	Nodes appear sequentially; connector lines draw; a pulse travels from trigger to action.
Simulation	Synthetic leads move along the graph; green passes, amber pauses, red blocks.
Risk report	Counters count upward, gauge settles into its risk tier, issues stagger into view.
Approval	Approval card expands to show the exact action; after approval, a checkmark draws and the card exits.
Audit trace	Events appear in sequence with status icons and timestamps.
Accessibility	Respect prefers-reduced-motion and provide a static status view for every animation.

10. API endpoints

Endpoint	Purpose
POST /api/compile	Instruction → validated workflow JSON
POST /api/simulate	Workflow → test results, traces, metrics, and issues
GET /api/report/:id	Return safety report for a simulation
POST /api/approvals/:id	Approve, reject, or edit a pending action
POST /api/execute/:id	Execute approved mock action and write audit events
GET /api/runs	List runs, states, and audit summaries

11. Demo storyline

1. Type a lead-follow-up instruction in plain language.
2. Watch the workflow graph form automatically.
3. Start the safety simulation.
4. See 20 synthetic leads travel through the workflow.
5. Observe missing-consent and duplicate cases stop before email sending.
6. Open the report: show pass rate, risk, confidence, and suggested fix.
7. Approve one safe email draft and show the audit trace.
8. Explain that FlowForge Guardian is a pre-deployment testing and governance layer, not a replacement for real customer or compliance review.

12. Definition of done

- A user can enter one natural-language workflow instruction.
- The system produces valid structured workflow JSON or a safe fallback demo workflow.

- The graph clearly shows trigger, checks, actions, and approval.
- At least six edge-case test records run in dry-run mode.
- The policy engine blocks at least two unsafe cases.
- The safety report explains failures in plain language.
- A human can approve or reject a pending action.
- Execution is mocked and fully logged.
- The UI works on desktop and tablet, with reduced-motion support.
- README documents architecture, setup, limitations, and synthetic-data disclosure.

Important implementation rule

Build the complete mocked vertical slice first: compile → simulate → report → approve → audit. Only add real integrations after this path works reliably. A polished, truthful prototype is stronger than a wide but fragile integration demo.